SỔ TAY ICPC — BẢN 13 TRANG KHỔ NGANG

Phong cách code: `#include<bits/stdc++.h>`, `using namespace std`, `long long`, `fast IO` | C++17 | Tiếng Việt | Bổ sung: đồ thị nâng cao, cây, DP, hình học, xác suất

Mục lục

1 BFS và các dạng đồ thị khó **3**
- 1.1 BFS ma trận có truy vết đường đi . . . 3
- 1.2 BFS 0–1 (cạnh trọng số 0/1) . . . 3
- 1.3 Dijkstra heap + dừng sớm . . . 3
- 1.4 BFS đa nguồn . . . 3
- 1.5 DSU + Kruskal (khó ở nén + rollback tư duy) . . . 4
- 1.6 LCA Binary Lifting . . . 4

2 Fenwick Tree (BIT) **4**
- 2.1 BIT cộng điểm – hỏi prefix/đoạn . . . 4
- 2.2 BIT range-add – point-query và range-add – range-sum . . . 4
- 2.3 BIT tìm chỉ số có prefix $\geq k$ (order statistics) . . . 4
- 2.4 BIT 2D (dạng khó) . . . 4

3 Segment Tree **4**
- 3.1 SegTree lazy cộng – hỏi max/sum . . . 4
- 3.2 Tìm phòng đầu tiên đủ chỗ (Hotel Queries) . . . 5
- 3.3 SegTree lặp (iterative) cho sum/min . . . 5

4 Trie **5**
- 4.1 Trie chuỗi thường (đếm prefix) . . . 5
- 4.2 Binary Trie – XOR lớn nhất (dạng khó) . . . 5
- 4.3 Aho–Corasick (Trie + KMP tập mẫu) . . . 5

5 KMP, Z, Hashing **6**
- 5.1 Hàm prefix π và KMP tìm mẫu . . . 6
- 5.2 Hàm Z . . . 6
- 5.3 Rolling Hash kép . . . 6
- 5.4 Manacher (palindrome dài nhất $O(n)$) . . . 6

6 Đệ quy có nhớ (memoization) **6**

7 DP chữ số (Digit DP) **6**
- 7.1 Kiểu (đếm, tổng) — hỏi TỔNG chữ số / tổng giá trị . . . 7
- 7.2 Ví dụ đầy đủ: tổng chữ số các số $\leq n$ và chia hết cho k . . . 7
- 7.3 Biến thể: trạng thái mang mặt nạ chữ số đã dùng . . . 7

8 DP Bitmask và các dạng khó **7**
- 8.1 TSP bitmask $O(n^2 2^n)$, $n \leq 20$ . . . 7
- 8.2 SOS DP (sum over subsets/supersets) . . . 7
- 8.3 DP phân công (assignment) $O(n2^n)$ . . . 8
- 8.4 Meet-in-the-middle ($n \leq 40$) . . . 8

9 Luồng cực đại **8**
9.1 Dinic ($O(V^2E)$, chạy nhanh thực tế) . . . 8
9.2 Hopcroft–Karp (ghép cặp cực đại bipartite) . . . 8
9.3 Min-Cost Max-Flow (potentials + Dijkstra) . . . 8

10 Local Search: Hill Climbing **9**
10.1 Hill Climbing (tô màu / lập lịch / TSP) . . . 9

11 Phụ lục: Số học và tổ hợp hay dùng **9**
11.1 Lũy thừa mod, nghịch đảo, nCr . . . 9
11.2 Đại số modulo . . . 9
11.3 Sàng tuyến tính + phân tích thừa số . . . 9
11.4 Euclid mở rộng . . . 9

12 Đồ thị bổ sung: cầu–khớp, SCC, Euler **9**
12.1 Cầu (bridge) và khớp (articulation point) . . . 9
12.2 SCC – Kosaraju (2 lần DFS) . . . 9
12.3 Đường đi/chu trình Euler (Hierholzer) . . . 10

13 Đường đi ngắn bổ sung: Floyd – Bellman–Ford **10**
13.1 Floyd: mọi cặp đỉnh $O(n^3)$, $n \le 400$ . . . 10
13.2 Bellman–Ford: chu trình âm (High Score, Cycle Finding) . . . 10

14 Cây bổ sung: tree DP, rerooting, truy vấn **10**
14.1 Tree DP: khớp cặp lớn nhất trên cây (Tree Matching 1130) . . . 10
14.2 Rerooting: Tree Distances II (1133) . . . 10
14.3 Truy vấn cây con: Euler tour + BIT (Subtree Queries 1137) . . . 10
14.4 Khoảng cách, tổ tiên bậc k, diff đường đi . . . 10

15 Sparse table, nén tọa độ & BIT theo giá trị **10**
15.1 Sparse table: RMQ $O(1)$, xây $O(n \log n)$ . . . 10
15.2 Nén tọa độ . . . 10
15.3 BIT max cho DP trên giá trị nén (Increasing Subsequence II, Projects) . . . 11
15.4 BIT đếm theo giá trị (Salary Queries 1144) . . . 11

16 Xác suất & kỳ vọng cơ bản **11**

17 Game đối kháng: Grundy (mex) **11**

18 Hình học 2D cơ bản **11**
18.1 Giao hai đoạn thẳng (kể cả chạm đầu mút) . . . 11
18.2 Diện tích đa giác & điểm trong đa giác . . . 11
18.3 Bao lồi – monotone chain . . . 11

19 Tiện ích bổ sung **12**
19.1 Prefix 3D (dạng đã gặp ở ICPC2025MB) . . . 12
19.2 Bitset: subset sum nhanh gấp 64 lần . . . 12
19.3 Chuỗi: chu kỳ & LCP bằng hash . . . 12
19.4 ordered_set (PBDS) . . . 12

20 DP bổ sung: interval, knapsack, bitmask, tối ưu **12**
20.1 Interval DP: dp[l][r] . . . 12
20.2 Tối ưu bằng prefix sum: $O(n^2) \to O(n)$ . . . 12
20.3 Knapsack: các biến thể . . . 12
20.4 LIS $O(n \log n)$ và biến thể . . . 12
20.5 DP chuỗi: wildcard ký tự * ? . . . 13
20.6 Bitmask bổ sung . . . 13
20.7 DP cửa sổ trượt bằng deque . . . 13
20.8 Max subarray 2D — Kadane nâng cấp $O(n^3)$ . . . 13
20.9 Tree DP nhỏ: đường kính qua mỗi đỉnh . . . 13
20.10 Đếm đường đi tránh vật cản (tổ hợp + đệ quy) . . . 13
20.11 Broken profile DP: lấp domino 1×2 (Counting Tilings 2181) . . . 13

1. BFS và các dạng đồ thị khó

Phần này gồm: BFS lưới có truy vết, BFS 0–1, Dijkstra heap, BFS đa nguồn, DSU + Kruskal, LCA.

1.1 BFS ma trận có truy vết đường đi

Ý tưởng: đánh dấu hướng đi tới mỗi ô, sau đó lội ngược từ B về A (bài toán mê cung, CSES 1193). Bảng `mask` lưu ký tự hướng đã dùng để tới ô đó.

```
// BFS luoi n x m, A -> B, vat can '#'. Tra ve chuoi duong L/R/U/D
string bfs_grid(vector<string>& a){
  int n=a.size(), m=a[0].size();
  pair<int,int> s{-1,-1}, t{-1,-1};
  for(int i=0;i<n;i++) for(int j=0;j<m;j++){
    if(a[i][j]=='A') s={i,j};
    if(a[i][j]=='B') t={i,j};
  }
  vector<string> mask(n, string(m,'?'));
  queue<pair<int,int>> q;
  q.push(s); mask[s.first][s.second]='S';
  int dx[4]={0,0,-1,1}; int dy[4]={-1,1,0,0};
  char dc[4]={'L','R','U','D'};
  bool found=false;
  while(!q.empty() && !found){
    auto [x,y]=q.front(); q.pop();
    for(int k=0;k<4;k++){
      int nx=x+dx[k], ny=y+dy[k];
      if(nx<0||nx>=n||ny<0||ny>=m) continue;
      if(a[nx][ny]=='#'||mask[nx][ny]!='?') continue;
      mask[nx][ny]=dc[k];
      q.push({nx,ny});
      if(nx==t.first&&ny==t.second){found=true;break;}
    }
  }
  if(!found) return "";
  string ans="";
  int x=t.first,y=t.second;
  unordered_map<char,pair<int,int>> d={
    {'L',{0,-1}},{'R',{0,1}},{'U',{-1,0}},{'D',{1,0}}};
  while(mask[x][y]!='S'){
    char c=mask[x][y]; ans+=c;
    x-=d[c].first; y-=d[c].second;
  }
  reverse(ans.begin(),ans.end());
  return ans;
}
```

1.2 BFS 0–1 (cạnh trọng số 0/1)

Dùng `deque`. Bài toán biến đổi tốn 0/1, lưới có chi phí vào ô.

```
vector<int> bfs01(vector<vector<pair<int,int>>>& adj,int s){
  int n=adj.size(); vector<int> d(n,INF);
  deque<int> dq; d[s]=0; dq.push_front(s);
  while(!dq.empty()){
    int u=dq.front(); dq.pop_front();
    for(auto [v,w]:adj[u]){
      if(d[v]>d[u]+w){ d[v]=d[u]+w;
        if(w==0) dq.push_front(v); else dq.push_back(v);
      }
    }
  }
  return d;
}
```

1.3 Dijkstra heap + dừng sớm

```
vector<ll> dijkstra(vector<vector<pair<int,ll>>>& adj,int s){
  int n=adj.size(); vector<ll> d(n,INF64);
  priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<>> pq;
  d[s]=0; pq.push({0,s});
  while(!pq.empty()){
    auto [du,u]=pq.top(); pq.pop();
    if(du!=d[u]) continue;
    for(auto [v,w]:adj[u])
      if(d[v]>du+w){d[v]=du+w; pq.push({d[v],v});}
  }
  return d;
}
```

1.4 BFS đa nguồn

Đẩy mọi nguồn vào queue với `dist=0`. Dùng cho: lan truyền lửa/lây nhiễm, khoảng cách tới ô đặc biệt gần nhất, Voronoi trên lưới.

```
// Monsters (1194): nhieu quai M va nguoi A tren luoi, thoat ra bien.
// B1: BFS da nguon tu moi M -> dm[][]. B2: BFS tu A, chi buoc vao o
// khi den som hon quai (den cung luc la thua). Cham bien -> thoat duoc.
int dm[N][N], ds[N][N]; // -1 = chua tham
queue<pair<int,int>> q;
for i,j: if(a[i][j]=='M'){ dm[i][j]=0; q.push({i,j}); }
while(!q.empty()){
  auto [x,y]=q.front(); q.pop();
  for(int k=0;k<4;k++){ int nx=x+dx[k],ny=y+dy[k];
```

```
    if(nx<0||ny<0||nx>=n||ny>=m||a[nx][ny]=='#'||dm[nx][ny]!=-1) continue;
    dm[nx][ny]=dm[x][y]+1; q.push({nx,ny});
  }
}
// B2 tuong tu, them dieu kien ds[x][y]+1 < dm[nx][ny] moi duoc vao o
```

1.5 DSU + Kruskal (khó ở nén + rollback tư duy)

```
struct DSU{
  vector<int> p,r;
  DSU(int n=0){init(n);}
  void init(int n){p.resize(n);r.assign(n,0);iota(p.begin(),p.end(),0);}
  int find(int x){return p[x]==x?x:p[x]=find(p[x]);}
  bool unite(int a,int b){
    a=find(a);b=find(b); if(a==b) return false;
    if(r[a]<r[b]) swap(a,b);
    p[b]=a; if(r[a]==r[b]) r[a]++; return true;
  }
};
```

1.6 LCA Binary Lifting

Tiền xử lý $O(n \log n)$, truy vấn $O(\log n)$. Cần cho bài đường đi + max/min cạnh.

```
const int LOG=20;
int n; vector<vector<int>> up; vector<int> dep;
vector<vector<pair<int,int>>> tree;
void dfs_lca(int u,int p){
  up[u][0]=p;
  for(int k=1;k<LOG;k++) up[u][k]=up[up[u][k-1]][k-1];
  for(auto [v,w]:tree[u]) if(v!=p){dep[v]=dep[u]+1;dfs_lca(v,u);}
}
int lca(int a,int b){
  if(dep[a]<dep[b]) swap(a,b);
  int d=dep[a]-dep[b];
  for(int k=0;k<LOG;k++) if(d>>k&1) a=up[a][k];
  if(a==b) return a;
  for(int k=LOG-1;k>=0;k--) if(up[a][k]!=up[b][k]){a=up[a][k];b=up[b][k];}
  return up[a][0];
}
```

2. Fenwick Tree (BIT)

Nhẹ hơn Segment Tree, code ngắn, ít bug. Nhớ: BIT gốc chỉ làm prefix; muốn đoạn phải biến đổi.

2.1 BIT cộng điểm – hỏi prefix/đoạn

```
struct BIT{
  int n; vector<ll> bit;
  BIT(int n=0):n(n),bit(n+1,0){}
  void add(int idx,ll val){ // 0-indexed
    for(int i=idx+1;i<=n;i+=i&-i) bit[i]+=val;
  }
  ll sumPrefix(int idx){ // sum [0..idx], idx<0 -> 0
    ll s=0;
    for(int i=idx+1;i>0;i-=i&-i) s+=bit[i];
    return s;
  }
  ll rangeSum(int l,int r){
    if(l>r) return 0;
    return sumPrefix(r)-(l?sumPrefix(l-1):0);
  }
};
```

2.2 BIT range-add – point-query và range-add – range-sum

Cộng v vào $[l,r]$: `add(l,v); add(r+1,-v)` rồi hỏi điểm bằng prefix. Muốn hỏi tổng đoạn sau khi cộng đoạn: dùng 2 BIT.

```
struct BITRange{
  int n; BIT b1,b2;
  BITRange(int n=0):n(n),b1(n+1),b2(n+1){}
  void _add(BIT& b,int idx,ll v){ if(idx<=n) b.add(idx,v); }
  // cong v vao [l,r]
  void rangeAdd(int l,int r,ll v){
    _add(b1,l,v); _add(b1,r+1,-v);
    _add(b2,l,v*(l-1)); _add(b2,r+1,-v*r);
  }
  ll prefixSum(int idx){ // tong [0..idx]
    return b1.sumPrefix(idx)*idx - b2.sumPrefix(idx);
  }
  ll rangeSum(int l,int r){ return prefixSum(r)-(l?prefixSum(l-1):0); }
};
```

2.3 BIT tìm chỉ số có prefix $\geq k$ (order statistics)

Dùng cho: xóa phần tử thứ k, Hotel Queries bản đơn giản, phân phối kẹo. $k \geq 1$.

```
// bit: BIT tan suat (0/1 hoac dem). Tim idx nho nhat sao cho prefix>=k
int kth(BIT& bit,int k){
  int idx=0;
  int pw=1; while((pw<<1)<=bit.n) pw<<=1;
  for(int d=pw;d>0;d>>=1){
    int nxt=idx+d;
    if(nxt<=bit.n && bit.bit[nxt]<k){k-=bit.bit[nxt]; idx=nxt;}
  }
  return idx; // 0-indexed: idx (vi bit 1-indexed) -> dap an 0-based = idx
}
```

2.4 BIT 2D (dạng khó)

Cộng điểm (x,y), hỏi tổng hình chữ nhật. $n,m \leq 2000$: $O(\log n \log m)$.

```
struct BIT2D{
  int n,m; vector<vector<ll>> bit;
  BIT2D(int n=0,int m=0):n(n),m(m),bit(n+1,vector<ll>(m+1,0)){}
  void add(int x,int y,ll v){
    for(int i=x+1;i<=n;i+=i&-i) for(int j=y+1;j<=m;j+=j&-j) bit[i][j]+=v;
  }
  ll sum(int x,int y){
    ll s=0;
    for(int i=x+1;i>0;i-=i&-i) for(int j=y+1;j>0;j-=j&-j) s+=bit[i][j];
    return s;
  }
  ll rect(int x1,int y1,int x2,int y2){
    return sum(x2,y2)-sum(x1-1,y2)-sum(x2,y1-1)+sum(x1-1,y1-1);
  }
};
```

3. Segment Tree

Khi BIT không đủ (max/min, tìm vị trí đầu tiên $\geq v$, gán đoạn). Cài đặt đệ quy với `smt[4*n]` gồm `build/query/update`.

3.1 SegTree lazy cộng – hỏi max/sum

Đoạn code dưới dùng cho range-add + range-max. Muốn sum thì đổi `node=max` thành `sum` và nhân lazy với độ dài.

```
struct SegLazy{
  int n; vector<ll> st,lz;
  SegLazy(vector<ll>& a){n=a.size();st.assign(4*n,0);lz.assign(4*n,0);build(0,0,n-1,a);}
  void build(int i,int l,int r,vector<ll>& a){
    if(l==r){st[i]=a[l];return;}
    int m=(l+r)/2; build(2*i+1,l,m,a); build(2*i+2,m+1,r,a);
    st[i]=max(st[2*i+1],st[2*i+2]);
  }
  void push(int i){
    if(lz[i]!=0){
      for(int k:{2*i+1,2*i+2}){st[k]+=lz[i]; lz[k]+=lz[i];}
      lz[i]=0;
    }
  }
  void add(int i,int l,int r,int ql,int qr,ll v){
    if(qr<l||r<ql) return;
    if(ql<=l&&r<=qr){st[i]+=v; lz[i]+=v; return;}
    push(i); int m=(l+r)/2;
    add(2*i+1,l,m,ql,qr,v); add(2*i+2,m+1,r,ql,qr,v);
    st[i]=max(st[2*i+1],st[2*i+2]);
  }
  ll query(int i,int l,int r,int ql,int qr){
    if(qr<l||r<ql) return -INF64;
    if(ql<=l&&r<=qr) return st[i];
    push(i); int m=(l+r)/2;
    return max(query(2*i+1,l,m,ql,qr),query(2*i+2,m+1,r,ql,qr));
  }
  void add(int l,int r,ll v){add(0,0,n-1,l,r,v);}
  ll query(int l,int r){return query(0,0,n-1,l,r);}
};
```

3.2 Tìm phòng đầu tiên đủ chỗ (Hotel Queries)

Mỗi khách cần phòng có sức chứa $\geq p$, chọn phòng trái nhất, trừ đi và cập nhật (bài toán Hotel Queries, CSES 1143). Lặp m lần $O((n+m)\log n)$.

```
void build_mx(vector<ll>& smt,vector<ll>& a,int i,int l,int r){
  if(l==r){smt[i]=a[l];return;}
  int m=(l+r)/2; build_mx(smt,a,2*i+1,l,m); build_mx(smt,a,2*i+2,m+1,r);
  smt[i]=max(smt[2*i+1],smt[2*i+2]);
}
int find_first(vector<ll>& smt,int i,int l,int r,ll val){
  if(smt[i]<val) return -1;
  if(l==r) return l;
  int m=(l+r)/2;
  if(smt[2*i+1]>=val) return find_first(smt,2*i+1,l,m,val);
  return find_first(smt,2*i+2,m+1,r,val);
}
void point_update(vector<ll>& smt,int i,int l,int r,int pos,ll v){
  if(l==r){smt[i]=v;return;}
  int m=(l+r)/2;
  if(pos<=m) point_update(smt,2*i+1,l,m,pos,v);
  else point_update(smt,2*i+2,m+1,r,pos,v);
  smt[i]=max(smt[2*i+1],smt[2*i+2]);
}
```

3.3 SegTree lặp (iterative) cho sum/min

Ngắn, nhanh, ít tràn stack. Phù hợp khi n tới $2 \cdot 10^5$ và query dày.

```
struct SegIter{
  int n; vector<ll> t; ll NEUT=0;
  SegIter(vector<ll>& a){
    n=1; while(n<(int)a.size()) n*=2;
    t.assign(2*n,NEUT);
    for(int i=0;i<(int)a.size();i++) t[n+i]=a[i];
    for(int i=n-1;i>0;i--) t[i]=t[2*i]+t[2*i+1];
  }
  void pointSet(int p,ll v){ p+=n; t[p]=v; for(p/=2;p;p/=2) t[p]=t[2*p]+t[2*p+1]; }
  ll rangeSum(int l,int r){ // [l,r]
    l+=n; r+=n; ll s=0;
    while(l<=r){ if(l%2==1) s+=t[l++]; if(r%2==0) s+=t[r--]; l/=2; r/=2; }
    return s;
  }
};
```

Muốn min/max: đổi `NEUT` thành `INF / -INF` và phép hợp thành `min/max`.

4. Trie

4.1 Trie chuỗi thường (đếm prefix)

Dùng mảng `next[26]` khi bảng chữ cái nhỏ; dùng `map` khi ký tự rộng. Nhớ khởi tạo `-1`.

```
struct Trie{
  vector<array<int,26>> nx; vector<int> cnt,end_;
  Trie(){nx.push_back({});nx[0].fill(-1);cnt={0};end_={0};}
  void insert(const string& s){
    int u=0; cnt[0]++;
    for(char ch:s){int c=ch-'a';
      if(nx[u][c]==-1){nx[u][c]=nx.size();nx.push_back({});nx.back().fill(-1);cnt.pb(0);end_.pb(0);}
      u=nx[u][c]; cnt[u]++;
    }
    end_[u]++;
  }
  int countPrefix(const string& p){
    int u=0; for(char ch:p){int c=ch-'a'; if(nx[u][c]==-1) return 0; u=nx[u][c];}
    return cnt[u];
  }
};
```

4.2 Binary Trie – XOR lớn nhất (dạng khó)

Chèn số, hỏi max xor với x. Dùng cho cặp xor max, truy vấn prefix xor.

```
struct BinTrie{
  struct Node{int ch[2];int cnt;Node(){ch[0]=ch[1]=-1;cnt=0;}};
  vector<Node> t; BinTrie(){t.emplace_back();}
  void insert(int x){
    int u=0; t[u].cnt++;
    for(int b=30;b>=0;b--){int bit=(x>>b)&1;
      if(t[u].ch[bit]==-1){t[u].ch[bit]=t.size();t.emplace_back();}
      u=t[u].ch[bit]; t[u].cnt++;
    }
  }
  int maxXor(int x){
    int u=0,ans=0;
    for(int b=30;b>=0;b--){int bit=(x>>b)&1,want=bit^1;
      if(t[u].ch[want]!=-1){ans|=1<<b;u=t[u].ch[want];}
      else u=t[u].ch[bit];
    }
    return ans;
  }
};
```

4.3 Aho–Corasick (Trie + KMP tập mẫu)

Kết hợp Trie và KMP để đếm số mẫu xuất hiện trong văn bản. $O(|text| + \sum |pat|)$.

```
struct Aho{
  struct Node{array<int,26> nx;int link;ll out;Node(){nx.fill(-1);link=0;out=0;}};
  vector<Node> t; Aho(){t.emplace_back();}
```

```
  void add(const string& s){int u=0;for(char ch:s){int c=ch-'a';
    if(t[u].nx[c]==-1){t[u].nx[c]=t.size();t.emplace_back();}u=t[u].nx[c];}t[u].out++;}
  void build(){
    queue<int> q;
    for(int c=0;c<26;c++){int v=t[0].nx[c];
      if(v!=-1){t[v].link=0;q.push(v);} else t[0].nx[c]=0;}
    while(!q.empty()){int u=q.front();q.pop();
      t[u].out+=t[t[u].link].out;
      for(int c=0;c<26;c++){int v=t[u].nx[c];
        if(v!=-1){t[v].link=t[t[u].link].nx[c];q.push(v);}
        else t[u].nx[c]=t[t[u].link].nx[c];}
    }
  }
  ll match(const string& s){int u=0;ll ans=0;for(char ch:s){u=t[u].nx[ch-'a'];ans+=t[u].out;}return ans
      ;}
};
```

5. KMP, Z, Hashing

5.1 Hàm prefix π và KMP tìm mẫu

```
vector<int> prefixFunc(const string& s){
  int n=s.size(); vector<int> pi(n,0);
  for(int i=1;i<n;i++){int j=pi[i-1];
    while(j>0&&s[i]!=s[j]) j=pi[j-1];
    if(s[i]==s[j]) j++; pi[i]=j;
  }
  return pi;
}
vector<int> kmp(const string& t,const string& p){
  string s=p+'#'+t; vector<int> pi=prefixFunc(s);
  vector<int> occ; int m=p.size();
  for(int i=m+1;i<(int)s.size();i++) if(pi[i]==m) occ.pb(i-2*m);
  return occ; // vi tri bat dau trong t
}
```

5.2 Hàm Z

`z[i]` = độ dài tiền tố khớp tại i. Dùng tìm mẫu, nén chuỗi, khớp cyclic shift.

```
vector<int> zFunc(const string& s){
  int n=s.size(); vector<int> z(n,0); int l=0,r=0;
  for(int i=1;i<n;i++){
    if(i<=r) z[i]=min(r-i+1,z[i-l]);
    while(i+z[i]<n&&s[z[i]]==s[i+z[i]]) z[i]++;
    if(i+z[i]-1>r){l=i;r=i+z[i]-1;}
  }
  return z;
}
```

5.3 Rolling Hash kép

Khi KMP/Z không đủ (so sánh chuỗi con bất kỳ, palindrome). Dùng 2 mod. Base nên là số lẻ lớn hơn 26 (có thể random); tránh base chẵn.

```
const int HMOD1=1000000007,HMOD2=1000000009,HBASE=911382323;
struct HashStr{
  string s; vector<ll> p1,p2,h1,h2;
  HashStr(const string& str=""){init(str);}
  void init(const string& str){s=str;int n=s.size();
    p1.assign(n+1,1);p2.assign(n+1,1);h1.assign(n+1,0);h2.assign(n+1,0);
    for(int i=0;i<n;i++){p1[i+1]=p1[i]*HBASE%HMOD1;p2[i+1]=p2[i]*HBASE%HMOD2;
      h1[i+1]=(h1[i]*HBASE+s[i])%HMOD1;h2[i+1]=(h2[i]*HBASE+s[i])%HMOD2;}}
  pair<ll,ll> get(int l,int r){ // [l,r]
    ll a=(h1[r+1]-h1[l]*p1[r-l+1]%HMOD1+HMOD1)%HMOD1;
    ll b=(h2[r+1]-h2[l]*p2[r-l+1]%HMOD2+HMOD2)%HMOD2;
    return {a,b};
  }
};
```

5.4 Manacher (palindrome dài nhất $O(n)$)

Thuật toán tuyến tính tìm mọi bán kính palindrome lẻ và chẵn.

```
// d1[i]: ban kinh palindrome le tam i (so o le), d2[i]: chan tam giua i-1,i
pair<vector<int>,vector<int>> manacher(const string& s){
  int n=s.size(); vector<int> d1(n),d2(n);
  for(int i=0,l=0,r=-1;i<n;i++){int k=(i>r)?1:min(d1[l+r-i],r-i+1);
    while(i-k>=0&&i+k<n&&s[i-k]==s[i+k]) k++; d1[i]=k--;
    if(i+k>r){l=i-k;r=i+k;}}
  for(int i=0,l=0,r=-1;i<n;i++){int k=(i>r)?0:min(d2[l+r-i+1],r-i+1);
    while(i-k-1>=0&&i+k<n&&s[i-k-1]==s[i+k]) k++; d2[i]=k--;
    if(i+k>r){l=i-k-1;r=i+k;}}
  return {d1,d2};
}
```

6. Đệ quy có nhớ (memoization)

Viết DP theo hướng đệ quy từ trạng thái lớn lùi về trạng thái cơ sở, mỗi trạng thái chỉ tính một lần. Hợp với bài có thứ tự tính tự nhiên (digit DP, game, đếm đường đi) mà không muốn tự sắp thứ tự vòng lặp.

```
// Khung chuan
vector<ll> memo; vector<int> vis; // memo.assign(n,-1) neu gia tri khong the am
ll dp(trang thai){
  if(dieu kien dung) return base;
  if(vis[id]) return memo[id];
  ll res=0;
  // ... chuyen trang thai, cong don vao res
  vis[id]=1; memo[id]=res;
  return res;
}
```

Bốn lưu ý hay gặp: (1) Chỉ nhớ trạng thái mà kết quả không phụ thuộc "đường tới". Ví dụ `tight` của digit DP phải nằm ngoài memo; `started` thì nhớ được vì nó là một phần trạng thái. (2) Nếu giá trị hợp lệ có thể là -1 thì dùng mảng `vis` riêng, đừng lấy `-1` làm dấu "chưa tính". (3) Nhiều chiều thì encode thành một số hoặc dùng `map/unordered_map` khi trạng thái thưa. (4) Đệ quy sâu (cỡ 10^6) dễ tràn stack — chuyển sang vòng lặp hoặc chạy trên môi trường stack lớn.

Ví dụ nhỏ — đếm cách phân tích n thành tổng các số nguyên dương không tăng (dạng "phân tích số"):

```
ll memo[1005][1005]; // fill -1 truoc
ll part(int n,int mx){ // phan tich n, cac phan <= mx
  if(n==0) return 1;
  if(memo[n][mx]!=-1) return memo[n][mx];
  ll r=0;
  for(int x=1;x<=min(n,mx);x++) r+=part(n-x,x);
  return memo[n][mx]=r;
}
// goi part(n,n)
```

7. DP chữ số (Digit DP)

Trạng thái gồm `pos, tight, started/leadingZero, sum/mod`. Đệ quy có memo khi `tight=0`.

```
string S; ll memo[20][200][2]; bool vis[20][200][2];
// dem so <= S co tong chu so == target (bo so 0? tu quy dinh)
```

```
ll dfs(int pos,int sum,bool tight,bool started,int target){
  if(pos==(int)S.size()) return (started&&sum==target)?1:0;
  if(!tight&&vis[pos][sum][started]) return memo[pos][sum][started];
  int lim=tight?S[pos]-'0':9; ll res=0;
  for(int d=0;d<=lim;d++){
    bool nstarted=started||(d!=0);
    int nsum=sum+(nstarted?d:0);
    if(nsum>target) continue;
    res+=dfs(pos+1,nsum,tight&&(d==lim),nstarted,target);
  }
  if(!tight){vis[pos][sum][started]=true; memo[pos][sum][started]=res;}
  return res;
}
```

Cách gọi: `S=to_string(R); memset(vis,0,sizeof vis); ans=dfs(0,0,true,false,target)`. Đếm đoạn $[L,R]$: `F(R)-F(L-1)`.

Ví dụ đếm số không chứa chữ số 4 và không chứa chuỗi 13 (bản đầy đủ):

```
string S; ll memo2[20][2]; bool vis2[20][2];
// prev1 = chu so ngay truoc co phai la 1 (de chan "13")
ll dfs2(int pos,bool tight,bool prev1){
  if(pos==(int)S.size()) return 1;
  if(!tight&&vis2[pos][prev1]) return memo2[pos][prev1];
  int lim=tight?S[pos]-'0':9; ll res=0;
  for(int d=0;d<=lim;d++){
    if(d==4) continue;          // khong chua 4
    if(prev1&&d==3) continue; // khong chua "13"
    res+=dfs2(pos+1,tight&&d==lim,d==1);
  }
  if(!tight){vis2[pos][prev1]=true; memo2[pos][prev1]=res;}
  return res;
}
// S=to_string(R); memset(vis2,0,sizeof vis2); dfs2(0,true,false)
// [L,R]: F(R)-F(L-1). (Mau tren khong dinh leading zero;
// neu dieu kien dinh so 0 thi can co co "started" nhu ham dfs o tren.)
```

7.1 Kiểu (đếm, tổng) — hỏi TỔNG chữ số / tổng giá trị

Bài "tính tổng chữ số của mọi số thỏa điều kiện": trạng thái trả về cặp (số lượng, tổng chữ số). Khi gắn thêm chữ số d vào mỗi số của nhánh con: `tong += tong_con + d * dem_con`. Nếu hỏi tổng giá trị của các số (không phải tổng chữ số): `tong += tong_con + d * 10^p * dem_con`, với p = số chữ số còn lại phía sau.

7.2 Ví dụ đầy đủ: tổng chữ số các số $\le n$ và chia hết cho k

Trạng thái `(pos, rem mod k, tight, started)`, trả về cặp (đếm, tổng chữ số). Với k nhỏ thì nhớ theo $O(k)$; với k lớn số bội số $\le n$ lại ít — chia hai nhánh theo ngưỡng. Chạy được $n \le 10^{12}$ với mọi $k \le n$.

```
const int LIM=100000; // k<=LIM: digit DP; k>LIM: duyet boi so
string S; int K,L;
ll mCnt[15][100005][2],mSum[15][100005][2];
bool vis[15][100005][2];

ll sumDigit(ll x){ ll s=0; while(x){ s+=x%10; x/=10; } return s; }

pair<ll,ll> dfs(int pos,int rem,bool tight,bool started){
  if(pos==L) return (started&&rem==0)?make_pair(1LL,0LL):make_pair(0LL,0LL);
  if(!tight&&vis[pos][rem][started])
    return make_pair(mCnt[pos][rem][started],mSum[pos][rem][started]);
  int lim=tight?S[pos]-'0':9;
  ll cnt=0,sum=0;
  for(int d=0;d<=lim;d++){
    auto [c,s]=dfs(pos+1,(rem*10+d)%K,tight&&d==lim,started||d);
    cnt+=c; sum+=s+1LL*d*c; // moi so trong nhanh con duoc them chu so d
  }
  if(!tight){vis[pos][rem][started]=1;
    mCnt[pos][rem][started]=cnt; mSum[pos][rem][started]=sum;}
  return make_pair(cnt,sum);
}
ll solve(ll n,ll k){
  if(k<=LIM){
    S=to_string(n); K=(int)k; L=S.size();
    memset(vis,0,sizeof vis);
    return dfs(0,0,true,false).second;
  }
  ll ans=0;
  for(ll x=k;x<=n;x+=k) ans+=sumDigit(x);
  return ans;
}
// main: while(cin>>n>>k) cout<<solve(n,k)<<"\n";
// DP: O(chuso * k * 10); duyet boi so: O(n/k * so chu so)
```

7.3 Biến thể: trạng thái mang mặt nạ chữ số đã dùng

Ví dụ đếm số $\le n$ mà các chữ số đôi một khác nhau — thêm `mask` 10 bit vào trạng thái:

```
string S; ll memoM[13][1024][2]; bool visM[13][1024][2];
ll dfsM(int pos,bool tight,bool started,int mask){
  if(pos==(int)S.size()) return started?1:0;
  if(!tight&&visM[pos][mask][started]) return memoM[pos][mask][started];
  int lim=tight?S[pos]-'0':9; ll res=0;
  for(int d=0;d<=lim;d++){
    if(started&&(mask>>d&1)) continue;      // chu so da xuat hien
    int nmask=(d==0&&!started)?0:(mask|(1<<d));
    res+=dfsM(pos+1,tight&&d==lim,started||d,nmask);
  }
  if(!tight){visM[pos][mask][started]=1; memoM[pos][mask][started]=res;}
  return res;
}
// Hoi so luong: tra ve dem (khong phai tong); hoi doan [L,R]: F(R)-F(L-1)
```

8. DP Bitmask và các dạng khó

Phần này gồm: TSP Held–Karp, SOS DP, phân công, đếm Hamiltonian.

8.1 TSP bitmask $O(n^2 2^n)$, $n \le 20$

```
ll tsp(vector<vector<ll>>& c){
  int n=c.size(), N=1<<n; const ll INFLL=INF64;
  vector<vector<ll>> dp(N,vector<ll>(n,INFLL));
  dp[1][0]=0;
  for(int mask=1;mask<N;mask++) for(int u=0;u<n;u++){
    if(!(mask&1<<u)||dp[mask][u]==INFLL) continue;
    for(int v=0;v<n;v++) if(!(mask&1<<v))
      dp[mask|1<<v][v]=min(dp[mask|1<<v][v],dp[mask][u]+c[u][v]);
  }
  ll ans=INFLL;
  for(int u=1;u<n;u++) ans=min(ans,dp[N-1][u]+c[u][0]);
  return ans;
}
```

8.2 SOS DP (sum over subsets/supersets)

$F[mask] = \sum_{sub \subseteq mask} A[sub]$. $O(k2^k)$.

```
void sos(vector<ll>& f,int k){
  int N=1<<k;
  for(int b=0;b<k;b++) for(int mask=0;mask<N;mask++)
    if(mask&1<<b) f[mask]+=f[mask^(1<<b)];
}
```

Dùng cho: đếm cặp (a,b) sao cho $a\&b == 0$, bao đóng tập, DP trên mặt nạ tần suất.

8.3 DP phân công (assignment) $O(n2^n)$

`dp[mask]` = chi phí min khi đã gán `popcount(mask)` việc đầu cho tập người `mask`.

```
ll assignMin(vector<vector<ll>>& a){
  int n=a.size(),N=1<<n; vector<ll> dp(N,INF64); dp[0]=0;
  for(int mask=0;mask<N;mask++){
    int i=__builtin_popcount(mask); // viec thu i
    for(int j=0;j<n;j++) if(!(mask&1<<j))
      dp[mask|1<<j]=min(dp[mask|1<<j],dp[mask]+a[i][j]);
  }
  return dp[N-1];
}
```

8.4 Meet-in-the-middle ($n \le 40$)

Chia đôi, liệt kê $2^{n/2}$ tổng mỗi nửa, sort + two-pointer / binary search. Dùng cho subset-sum, knapsack lớn, xor min.

```
// vi du: dem so tap co tong == S
ll meetCount(vector<ll>& a,ll S){
  int n=a.size(),n1=n/2; vector<ll> L,R;
  for(int m=0;m<1<<n1;m++){ll s=0;for(int i=0;i<n1;i++)if(m>>i&1)s+=a[i];L.pb(s);}
  int n2=n-n1;
  for(int m=0;m<1<<n2;m++){ll s=0;for(int i=0;i<n2;i++)if(m>>i&1)s+=a[n1+i];R.pb(s);}
  sort(R.begin(),R.end()); ll ans=0;
  for(ll x:L) ans+=upper_bound(R.begin(),R.end(),S-x)-lower_bound(R.begin(),R.end(),S-x);
  return ans;
}
```

9. Luồng cực đại

9.1 Dinic ($O(V^2E)$, chạy nhanh thực tế)

Mẫu cài đặt chuẩn: BFS level + DFS blocking flow + current arc.

```
struct Dinic{
  struct E{int to;ll cap;int rev;};
  int n; vector<vector<E>> g; vector<int> lv,it;
  Dinic(int n=0):n(n),g(n),lv(n),it(n){}
  void addEdge(int u,int v,ll c){E a{v,c,(int)g[v].size()};E b{u,0,(int)g[u].size()};g[u].pb(a);g[v].pb(
      b);}
  bool bfs(int s,int t){
    fill(lv.begin(),lv.end(),-1); queue<int> q; lv[s]=0; q.push(s);
    while(!q.empty()){int u=q.front();q.pop();
      for(auto &e:g[u]) if(e.cap>0&&lv[e.to]<0){lv[e.to]=lv[u]+1;q.push(e.to);}}
    return lv[t]>=0;
  }
  ll dfs(int u,int t,ll f){
    if(u==t) return f;
    for(int &i=it[u];i<(int)g[u].size();i++){
      E &e=g[u][i];
      if(e.cap>0&&lv[u]<lv[e.to]){
        ll d=dfs(e.to,t,min(f,e.cap));
        if(d>0){e.cap-=d; g[e.to][e.rev].cap+=d; return d;}
      }
    }
    return 0;
  }
  ll maxflow(int s,int t){ll flow=0,INF_F=INF64;
    while(bfs(s,t)){fill(it.begin(),it.end(),0);ll f;
      while((f=dfs(s,t,INF_F))>0) flow+=f;}
    return flow;
  }
};
```

9.2 Hopcroft–Karp (ghép cặp cực đại bipartite)

$O(E\sqrt{V})$. Dùng khi Dinic quá chậm cho bipartite 10^5.

```
struct HopcroftKarp{
  int nL,nR; vector<vector<int>> adj; vector<int> dist,pu,pv;
  HopcroftKarp(int L,int R):nL(L),nR(R),adj(L),dist(L),pu(L,-1),pv(R,-1){}
  void add(int u,int v){adj[u].pb(v);}
  bool bfs(){
    queue<int> q;
    for(int u=0;u<nL;u++){ if(pu[u]==-1){dist[u]=0;q.push(u);} else dist[u]=-1;}
    bool found=false;
    while(!q.empty()){int u=q.front();q.pop();
      for(int v:adj[u]){int w=pv[v];
        if(w!=-1&&dist[w]==-1){dist[w]=dist[u]+1;q.push(w);}
        if(w==-1) found=true;}}
    return found;
  }
  bool dfs(int u){
    for(int v:adj[u]){int w=pv[v];
      if(w==-1||(dist[w]==dist[u]+1&&dfs(w))){pu[u]=v;pv[v]=u;return true;}}
    dist[u]=-1; return false;
  }
  int maxmatch(){int m=0;
    while(bfs()){for(int u=0;u<nL;u++) if(pu[u]==-1&&dfs(u)) m++;}
    return m;
  }
};
```

9.3 Min-Cost Max-Flow (potentials + Dijkstra)

Dùng cho gán chi phí min, vận chuyển. $n \le 500$.

```
struct MCMF{
  struct E{int to,rev;ll cap,cost;};
  int n; vector<vector<E>> g;
  MCMF(int n=0):n(n),g(n){}
  void addEdge(int u,int v,ll cap,ll cost){
    E a{v,(int)g[v].size(),cap,cost},b{u,(int)g[u].size(),0,-cost};
    g[u].pb(a);g[v].pb(b);}
  pair<ll,ll> minCost(int s,int t,ll maxf=INF64){
    ll flow=0,cost=0; vector<ll> h(n,0),d(n); vector<int> pv(n),pe(n);
    while(flow<maxf){
      priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<>> pq;
      fill(d.begin(),d.end(),INF64); d[s]=0; pq.push({0,s});
      while(!pq.empty()){auto [du,u]=pq.top();pq.pop();
        if(du!=d[u]) continue;
        for(int i=0;i<(int)g[u].size();i++){E &e=g[u][i];
          if(e.cap>0&&d[e.to]>du+e.cost+h[u]-h[e.to]){
            d[e.to]=du+e.cost+h[u]-h[e.to];pv[e.to]=u;pe[e.to]=i;pq.push({d[e.to],e.to});}}}
      if(d[t]==INF64) break;
      for(int i=0;i<n;i++) if(d[i]<INF64) h[i]+=d[i];
      ll add=maxf-flow;
      for(int v=t;v!=s;v=pv[v]) add=min(add,g[pv[v]][pe[v]].cap);
      for(int v=t;v!=s;v=pv[v]){E &e=g[pv[v]][pe[v]];e.cap-=add;g[v][e.rev].cap+=add;}
      flow+=add; cost+=add*h[t];
    }
    return {flow,cost};
  }
};
```

10. Local Search: Hill Climbing

Dùng khi bài NP-khó cần điểm từng phần (TSP, sắp xếp, tô màu, lập lịch). Khung chung: biểu diễn nghiệm + hàm delta + vòng lặp có giới hạn giờ.

10.1 Hill Climbing (tô màu / lập lịch / TSP)

```
// hill climbing co khoi dong lai (random restart)
template<class State,class Cost,class Neighbor>
State hillClimb(State s,Cost cost,Neighbor neigh,int iters=200000,int restarts=20){
  mt19937 r2(12345); State best=s; auto bestC=cost(s);
  for(int rs=0;rs<restarts;rs++){
    State cur=(rs==0?s:neigh(s,true)); auto curC=cost(cur);
    for(int it=0;it<iters;it++){
      State ns=neigh(cur,false); auto nc=cost(ns);
      if(nc<curC){cur=ns;curC=nc; if(nc<bestC){best=ns;bestC=nc;}}
      // tabu don gian: cam quay lai trang thai vua gap bang unordered_set hash
    }
  }
  return best;
}
```

11. Phụ lục: Số học và tổ hợp hay dùng

11.1 Lũy thừa mod, nghịch đảo, nCr

```
const ll MOD=1e9+7;
ll modpow(ll a,ll e,ll mod=MOD){ll r=1;while(e){if(e&1)r=r*a%mod;a=a*a%mod;e>>=1;}return r;}
// nCr nho voi giai thua san (n<=2e6)
vector<ll> fact,invFact;
void initFact(int n){fact.assign(n+1,1);invFact.assign(n+1,1);
  for(int i=1;i<=n;i++) fact[i]=fact[i-1]*i%MOD;
  invFact[n]=modpow(fact[n],MOD-2); for(int i=n;i>0;i--) invFact[i-1]=invFact[i]*i%MOD;}
ll Cnk(int n,int k){if(k<0||k>n) return 0; return fact[n]*invFact[k]%MOD*invFact[n-k]%MOD;}
```

11.2 Đại số modulo

Mọi phép toán đưa về $[0, MOD)$. Nhớ $a-b$ có thể âm nên cộng MOD; $a \cdot b$ tràn 64-bit khi $MOD \sim 10^9$ thì dùng `__int128`. Chia modulo = nhân nghịch đảo (chỉ khi MOD nguyên tố và a không chia hết cho MOD).

```
// MOD da dinh nghia o muc tren
inline ll addmod(ll a,ll b){ll s=a+b; if(s>=MOD) s-=MOD; return s;}
inline ll submod(ll a,ll b){ll s=a-b; if(s<0) s+=MOD; return s;}
inline ll mulmod(ll a,ll b){return (ll)((__int128)(a%MOD)*(b%MOD)%MOD);}
ll modpow(ll a,ll e,ll mod=MOD){ll r=1%mod; a%=mod; while(e){if(e&1)r=mulmod(r,a); a=mulmod(a,a); e
    >>=1;} return r;}
ll modInv(ll a){return modpow(a,MOD-2);} // MOD nguyen to
ll modDiv(ll a,ll b){return mulmod(a,modInv(b));}
// nghich dao 1..n tuyen tinh O(n) (MOD nguyen to)
vector<ll> buildInv(int n){vector<ll> inv(n+1); inv[1]=1;
  for(int i=2;i<=n;i++) inv[i]=MOD-MOD/i*inv[MOD%i]%MOD; return inv;}
// tong cap so nhan 1+a+...+a^{n-1} mod MOD
ll geomSum(ll a,ll n){
  if(n==0) return 0; if(n==1) return 1;
  if(n%2==0){ll h=geomSum(a,n/2); return mulmod(h,addmod(1,modpow(a,n/2)));}
  return addmod(geomSum(a,n-1),modpow(a,n-1));
}
ll Pnk(int n,int k){if(k<0||k>n) return 0; ll r=1; for(int i=0;i<k;i++) r=mulmod(r,n-i); return r;}
```

11.3 Sàng tuyến tính + phân tích thừa số

```
vector<int> primes,lp;
void sieve(int N){lp.assign(N+1,0);
  for(int i=2;i<=N;i++){if(lp[i]==0){lp[i]=i;primes.pb(i);}
    for(int p:primes){if(p>lp[i]||1LL*i*p>N) break; lp[i*p]=p;}}}
```

11.4 Euclid mở rộng

```
ll exgcd(ll a,ll b,ll &x,ll &y){
  if(b==0){x=1;y=0;return a;}
  ll x1,y1; ll g=exgcd(b,a%b,x1,y1); x=y1; y=x1-y1*(a/b); return g;
}
```

12. Đồ thị bổ sung: cầu–khớp, SCC, Euler

Ba dạng chuẩn ICPC mà bản đầu chưa có: cầu/khớp $O(n+m)$, thành phần liên thông mạnh, đường đi Euler.

12.1 Cầu (bridge) và khớp (articulation point)

DFS một lần: `num[u]` = thứ tự thăm, `low[u]` = num nhỏ nhất vươn tới được từ cây con u. Cạnh (u, v) trên cây DFS là cầu khi `low[v] > num[u]`; u khác gốc là khớp khi `low[v] >= num[u]`. Gốc là khớp khi có ≥ 2 con.

```
int timer; vector<int> num,low,isCut;
vector<vector<int>> g; vector<pair<int,int>> bridges;
void dfs_cut(int u,int par){
  num[u]=low[u]=++timer; int child=0;
  for(int v:g[u]){
    if(v==par) continue;
    if(num[v]) low[u]=min(low[u],num[v]);
    else{
      dfs_cut(v,u); child++;
      low[u]=min(low[u],low[v]);
      if(low[v]>num[u]) bridges.pb({u,v});
      if(par!=-1&&low[v]>=num[u]) isCut[u]=1;
    }
  }
  if(par==-1&&child>1) isCut[u]=1;
}
// timer=0; num/low/isCut gan (n+1,0); for i=1..n: if(!num[i]) dfs_cut(i,-1);
// Canh song song: thay v==par bang so hieu canh (khong bo qua theo dinh)
```

12.2 SCC – Kosaraju (2 lần DFS)

Lần 1 lấy thứ tự kết thúc của DFS; lần 2 DFS trên đồ thị ngược theo thứ tự đảo. Sau khi có `comp[u]` (0..c-1), nén thành DAG rồi DP (Coin Collector).

```
vector<vector<int>> g,rg; vector<int> vis,comp,order;
void dfs1(int u){ vis[u]=1;
  for(int v:g[u]) if(!vis[v]) dfs1(v);
  order.pb(u);
}
void dfs2(int u,int c){ comp[u]=c;
  for(int v:rg[u]) if(comp[v]==-1) dfs2(v,c);
}
int scc(int n){
  vis.assign(n+1,0); comp.assign(n+1,-1); order.clear();
  for(int i=1;i<=n;i++) if(!vis[i]) dfs1(i);
  reverse(order.begin(),order.end());
  int c=0;
  for(int u:order) if(comp[u]==-1) dfs2(u,c++);
  return c;
}
// Nen DAG: voi moi canh u->v ma comp[u]!=comp[v]: add canh comp[u]->comp[v]
```

12.3 Đường đi/chu trình Euler (Hierholzer)

Điều kiện: liên thông (bỏ đỉnh cô lập) và số đỉnh bậc lẻ bằng 0 (chu trình) hoặc đúng 2 (đường đi, xuất phát từ đỉnh bậc lẻ). Đồ thị có hướng dùng bán bậc ra–vào. Dùng số hiệu cạnh để xử lý cạnh song song/tự khuyên.

```
struct E{int to,id;};
vector<vector<E>> g; vector<int> usedE,path; // path = thu tu canh di qua
void dfs_euler(int u){
  for(auto [v,id]:g[u]) if(!usedE[id]){
    usedE[id]=1; dfs_euler(v); path.pb(id);
  }
}
// goi tu dinh bac le (neu co); reverse(path) de in dung thu tu
```

13. Đường đi ngắn bổ sung: Floyd – Bellman–Ford

13.1 Floyd: mọi cặp đỉnh $O(n^3)$, $n \le 400$

```
// d[i][j]=INF64 neu khong co canh, d[i][i]=0
for(int k=1;k<=n;k++) for(int i=1;i<=n;i++) for(int j=1;j<=n;j++)
  if(d[i][k]<INF64&&d[k][j]<INF64) d[i][j]=min(d[i][j],d[i][k]+d[k][j]);
```

13.2 Bellman–Ford: chu trình âm (High Score, Cycle Finding)

```
vector<ll> d(n+1,INF64); d[s]=0;
for(int it=1;it<n;it++)
  for(auto [u,v,w]:edges) if(d[u]<INF64) d[v]=min(d[v],d[u]+w);
vector<int> bad(n+1,0);
for(int it=0;it<n;it++)
  for(auto [u,v,w]:edges)
    if(d[u]<INF64&&d[v]>d[u]+w){ d[v]=d[u]+w; bad[v]=1; }
for(int it=0;it<n;it++)
  for(auto [u,v,w]:edges) if(bad[u]) bad[v]=1; // lan truyen
// bad[x]=1: duong tu s toi x cham chu trinh am (High Score: bad[n] -> in -1)
```

14. Cây bổ sung: tree DP, rerooting, truy vấn

14.1 Tree DP: khớp cặp lớn nhất trên cây (Tree Matching 1130)

dp0[u] = không ghép cạnh nào từ u xuống con; dp1[u] = được phép ghép tối đa 1 con (luôn có `dp1 >= dp0`).

```
ll dp0[N],dp1[N];
void dfs(int u,int p){
  dp0[u]=0; ll add=0;
  for(int v:g[u]) if(v!=p){
    dfs(v,u);
    dp0[u]+=dp1[v];
    add=max(add,1ll+dp0[v]-dp1[v]);
  }
  dp1[u]=dp0[u]+add;
}
// dap an: dp1[1]
```

14.2 Rerooting: Tree Distances II (1133)

```
// sz[u] = so dinh cay con; down[u] = tong khoang cach trong cay con tai u
int sz[N]; ll down[N],ans[N];
void dfs1(int u,int p){
  sz[u]=1; down[u]=0;
  for(int v:g[u]) if(v!=p){
    dfs1(v,u); sz[u]+=sz[v]; down[u]+=down[v]+sz[v];
  }
}
void dfs2(int u,int p){
  for(int v:g[u]) if(v!=p){ ans[v]=ans[u]+n-2*sz[v]; dfs2(v,u); }
}
// goi dfs1(1,0); ans[1]=down[1]; dfs2(1,0);
```

14.3 Truy vấn cây con: Euler tour + BIT (Subtree Queries 1137)

```
int tin[N],tout[N],timer=0;
void dfs_euler(int u,int p){
  tin[u]=++timer;
  for(int v:g[u]) if(v!=p) dfs_euler(v,u);
  tout[u]=timer;
}
// cong them v: bit.add(tin[u],v); tong cay con u: bit.rangeSum(tin[u],tout[u])
```

14.4 Khoảng cách, tổ tiên bậc k, diff đường đi

Dùng chung `up[u][k]`, `dep[]` với mục LCA ở trên. Binary lifting còn dùng cho đồ thị chức năng (mỗi đỉnh đúng 1 cạnh ra, Planets Queries).

```
// dep dem canh; neu canh co trong so: pre[u] = tong trong so tu goc
int dist(int u,int v){ return dep[u]+dep[v]-2*dep[lca(u,v)]; }
int kth(int u,int k){ // to tien thu k (0 = chinh u)
  for(int b=0;b<LOG;b++) if(k>>b&1) u=up[u][b];
  return u;
}
// Diff tren cay (Counting Paths 1136) - cong don len (val[u] += val[con]):
//  dem theo CANH: val[u]++, val[v]++, val[lca(u,v)] -= 2;
//  dem theo DINH: val[u]++, val[v]++, val[lca]--, val[parent(lca)]--;
```

15. Sparse table, nén tọa độ & BIT theo giá trị

15.1 Sparse table: RMQ $O(1)$, xây $O(n \log n)$

```
struct SparseTable{
  int n; vector<int> lg; vector<vector<int>> st;
  SparseTable(vector<int>& a){
    n=a.size(); lg.assign(n+1,0);
    for(int i=2;i<=n;i++) lg[i]=lg[i/2]+1;
    st.assign(lg[n]+1,vector<int>(n)); st[0]=a;
    for(int k=1;k<(int)st.size();k++)
      for(int i=0;i+(1<<k)<=n;i++)
        st[k][i]=min(st[k-1][i],st[k-1][i+(1<<(k-1))]);
  }
  int rmq(int l,int r){ // [l,r]
    int k=lg[r-l+1];
    return min(st[k][l],st[k][r-(1<<k)+1]);
  }
};
// max: doi min thanh max
```

15.2 Nén tọa độ

```
vector<int> val=a; // copy tat ca gia tri can nen
sort(val.begin(),val.end());
val.erase(unique(val.begin(),val.end()),val.end());
int id(int x){ return lower_bound(val.begin(),val.end(),x)-val.begin(); }
```

15.3 BIT max cho DP trên giá trị nén (Increasing Subsequence II, Projects)

```
// dp[i] = 1 + max dp[j] voi a[j] < a[i]. BIT chi can prefix max
struct BITMax{
  int n; vector<ll> bit;
  BITMax(int n=0):n(n),bit(n+1,0){}
  void upd(int i,ll v){ for(i++;i<=n;i+=i&-i) bit[i]=max(bit[i],v); }
  ll query(int i){ // max [0..i]
    ll r=0;
    for(i++;i>0;i-=i&-i) r=max(r,bit[i]);
    return r;
  }
};
// Projects: sort theo ngay ket thuc, nen ngay, dp = max(dp cu, query(id-1) + tien)
```

15.4 BIT đếm theo giá trị (Salary Queries 1144)

Nén cả giá trị lương xuất hiện trong truy vấn trước khi xử lý offline. Cập nhật: trừ ở giá trị cũ, cộng ở giá trị mới.

```
// dem.add(id(cu),-1); dem.add(id(moi),1);
// so nguoi luong trong [a,b]: l=lower_bound(val,a), r=upper_bound(val,b)-1
// (doi ra chi so nen) roi dem.rangeSum(l,r)
```

16. Xác suất & kỳ vọng cơ bản

Ba mẫu hay gặp: (1) số bước trung bình thỏa $E[u] = 1 + \sum p(u \to v)E[v]$ — đồ thị DAG thì tính ngược topo, có chu trình thì giải hệ; (2) xác suất lập mảng theo trạng thái rồi chuyển bước; (3) tuyến tính kỳ vọng $E[X+Y] = E[X] + E[Y]$ — tách bài toán lớn thành từng phần tử nhỏ, đếm xác suất rồi cộng lại.

```
// Throwing Dice: dp[s] = xac suat dat tong s
dp[0]=1;
for(int s=1;s<=n;s++)
  for(int k=1;k<=6;k++) if(s>=k) dp[s]+=dp[s-k]/6.0;
// Moving Robots: dp[o] = xac suat dang dung o o; lap tung buoc:
for(int step=0;step<k;step++){
  vector<double> nxt(n,0);
  for(int o=0;o<n;o++) for(int to:g[o]) nxt[to]+=dp[o]/g[o].size();
  swap(dp,nxt);
}
```

17. Game đối kháng: Grundy (mex)

Game gồm nhiều "đống"độc lập: g của hợp = XOR các g. Người đi trước thắng khi tổng XOR khác 0. $g[trngthi] = \text{mex}\{\, g[\text{trạng thái sau một nước đi hợp lệ}]\,\}$. Ví dụ: đống i viên, được bốc $mv \in moves$ viên.

```
vector<int> g(K+1,0);
for(int i=1;i<=K;i++){
  vector<int> seen(64,0); // 64 > so nuoc di
  for(int mv:moves) if(i>=mv) seen[g[i-mv]]=1;
  int mex=0; while(seen[mex]) mex++;
  g[i]=mex;
}
// Nhieu dong: ans = g[a1]^g[a2]^... (khac 0: nguoi di truoc thang)
```

18. Hình học 2D cơ bản

Tọa độ nguyên thì dùng `ll` với cross/dot, tránh chia và tránh double. Quy ước: `cross(b-a,c-a) > 0` khi c nằm bên trái hướng $a \to b$ (ngược chiều kim đồng hồ).

```
struct P{ ll x,y; };
P operator+(P a,P b){ return {a.x+b.x,a.y+b.y}; }
P operator-(P a,P b){ return {a.x-b.x,a.y-b.y}; }
ll dot(P a,P b){ return a.x*b.x+a.y*b.y; }
ll cross(P a,P b){ return a.x*b.y-a.y*b.x; }
ll sgn(ll v){ return (v>0)-(v<0); }
// Dien tich tam giac (i,j,k) = abs(cross(j-i,k-i)); da giac = |tong cross 2 dinh lien tiep|/2
```

18.1 Giao hai đoạn thẳng (kể cả chạm đầu mút)

```
bool onSeg(P a,P b,P p){ // p tren ab (chi dung khi a,b,p thang hang)
  return min(a.x,b.x)<=p.x&&p.x<=max(a.x,b.x)
      && min(a.y,b.y)<=p.y&&p.y<=max(a.y,b.y);
}
bool segInter(P a,P b,P c,P d){
  ll o1=cross(b-a,c-a),o2=cross(b-a,d-a);
  ll o3=cross(d-c,a-c),o4=cross(d-c,b-c);
  if(sgn(o1)*sgn(o2)<0&&sgn(o3)*sgn(o4)<0) return true;
  if(o1==0&&onSeg(a,b,c)) return true;
  if(o2==0&&onSeg(a,b,d)) return true;
  if(o3==0&&onSeg(c,d,a)) return true;
  if(o4==0&&onSeg(c,d,b)) return true;
  return false;
}
```

18.2 Diện tích đa giác & điểm trong đa giác

```
// area2 = 2 x dien tich; dinh theo thu tu (thuan hoac nguoc)
ll area2(vector<P>& p){
  ll s=0; int n=p.size();
  for(int i=0;i<n;i++) s+=cross(p[i],p[(i+1)%n]);
  return abs(s);
}
// Diem trong da giac don (ray casting, so nguyen)
bool inPoly(vector<P>& p,P q){
  int n=p.size(); bool in=false;
  for(int i=0,j=n-1;i<n;j=i++){
    P a=p[i],b=p[j];
    if((a.y>q.y)!=(b.y>q.y)){
      ll lhs=(q.x-a.x)*(b.y-a.y), rhs=(q.y-a.y)*(b.x-a.x);
      if((b.y>a.y)?(lhs<rhs):(lhs>rhs)) in=!in;
    }
  }
  return in;
}
// Muon tinh ca bien: kiem tra onSeg voi tung canh truoc, dung lai
```

18.3 Bao lồi – monotone chain

```
vector<P> hull(vector<P> p){
  sort(p.begin(),p.end(),[](P a,P b){
    return a.x<b.x||(a.x==b.x&&a.y<b.y); });
  p.erase(unique(p.begin(),p.end(),[](P a,P b){
    return a.x==b.x&&a.y==b.y; }),p.end());
  if(p.size()<=2) return p;
  int n=p.size(),k=0; vector<P> h(2*n);
  for(int i=0;i<n;i++){
    while(k>=2&&cross(h[k-1]-h[k-2],p[i]-h[k-1])<=0) k--;
    h[k++]=p[i];
```

```
  }
  for(int i=n-2,t=k+1;i>=0;i--){
    while(k>=t&&cross(h[k-1]-h[k-2],p[i]-h[k-1])<=0) k--;
    h[k++]=p[i];
  }
  h.resize(k-1);
  return h;
}
// "<=0": bo diem thang hang tren bien; doi "<0" neu muon giu lai
```

19. Tiện ích bổ sung

19.1 Prefix 3D (dạng đã gặp ở ICPC2025MB)

```
// pre[i][j][k] = tong khoi [1..i][1..j][1..k]
for i 1..n: for j 1..m: for k 1..p:
  pre[i][j][k]=a[i][j][k]+pre[i-1][j][k]+pre[i][j-1][k]+pre[i][j][k-1]
    -pre[i-1][j-1][k]-pre[i-1][j][k-1]-pre[i][j-1][k-1]+pre[i-1][j-1][k-1];
// Tong khoi [x1..x2][y1..y2][z1..z2]: quy tac chan le, 8 so hang:
// +F(x2,y2,z2) -F(x1-1,y2,z2) -F(x2,y1-1,z2) -F(x2,y2,z1-1)
// +F(x1-1,y1-1,z2) +F(x1-1,y2,z1-1) +F(x2,y1-1,z1-1) -F(x1-1,y1-1,z1-1)
```

19.2 Bitset: subset sum nhanh gấp 64 lần

```
bitset<100001> dp; dp[0]=1;
for(int x:a) dp|=dp<<x;   // dp[s]=1 neu co tap con tong s
cout<<dp.count();         // dem so tong tao duoc
// Reachability do thi n<=2000: reach[u] la bitset cac dinh toi duoc tu u,
// xu ly nguoc topo: reach[u] |= reach[v] voi moi canh u->v
```

19.3 Chuỗi: chu kỳ & LCP bằng hash

```
// Chu ky nho nhat: p = n - pi[n-1] (pi = prefixFunc); la chu ky "dep" khi n%p==0
// LCP hai xau con (dung HashStr o tren): chat nhi phan do dai
int lcp(HashStr& h,int i,int j,int mx){
  int lo=0,hi=mx;
  while(lo<hi){ int mid=(lo+hi+1)/2;
    if(h.get(i,i+mid-1)==h.get(j,j+mid-1)) lo=mid;
    else hi=mid-1; }
  return lo;
}
```

19.4 ordered_set (PBDS)

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
typedef tree<int,null_type,less<int>,rb_tree_tag,
  tree_order_statistics_node_update> ordered_set;
// s.order_of_key(x): so phan tu < x
// *(s.find_by_order(k)): phan tu thu k (0-index)
// Muon luu trung nhau: dung pair<val,id>. Chi co tren g++ (CF, domjudge).
```

20. DP bổ sung: interval, knapsack, bitmask, tối ưu

Các dạng xếp từ sát trình hiện tại trở lên; dạng khó nhất là broken profile (cuối chương), học qua CSES 2181.

20.1 Interval DP: dp[l][r]

Trạng thái là một khoảng; chuyển bằng cách chọn điểm cắt hoặc rút hai đầu. Hai mẫu hay gặp:

Removal Game (1097) — hai người rút đầu/cuối, ai cũng tối ưu. Chỉ cần 2 lớp theo độ dài: $O(n^2)$ thời gian, $O(n)$ bộ nhớ.

```
// dp[l][r] = diem nguoi di truoc an duoc tu doan [l,r]
// dp[l][r] = max(a[l] + sum(l+1,r) - dp[l+1][r],
//                a[r] + sum(l,r-1) - dp[l][r-1])
vector<ll> pre(n+1,0),prev(n),cur(n);
for(int i=0;i<n;i++) pre[i+1]=pre[i]+a[i];
for(int len=1;len<=n;len++){
  for(int l=0;l+len<=n;l++){
    int r=l+len-1;
    if(l==r){ cur[l]=a[l]; continue; }
    cur[l]=max(a[l]+pre[r+1]-pre[l+1]-prev[l+1],
               a[r]+pre[r]-pre[l]-prev[l]);
  }
  swap(prev,cur); // lop do dai len-1 -> len
}
// dap an: prev[0]
```

Gộp đoạn: $dp[l][r] = \min_k(dp[l][k] + dp[k+1][r]) + \text{cost}(l,r)$ (nối đá, nối xâu, nhân ma trận):

```
for(int len=2;len<=n;len++)
  for(int l=0;l+len<=n;l++){
    int r=l+len-1; dp[l][r]=INF64;
    for(int k=l;k<r;k++)
      dp[l][r]=min(dp[l][r],dp[l][k]+dp[k+1][r]);
    dp[l][r]+=pre[r+1]-pre[l]; // vi du: cost = tong doan
  }
// Xau doi xung: LPS = LCS(s, nguoc(s)); palindrome chan/le = Manacher (muc truoc)
```

20.2 Tối ưu bằng prefix sum: $O(n^2) \to O(n)$

```
// dp[i] = tong dp[j] voi j trong [i-k, i-1] -> prefix sum cua dp
// Vi du Dice: dem day dai n, moi buoc cong 1..k
vector<ll> dp(n+1),pre(n+2);
dp[0]=1; pre[1]=1;
for(int i=1;i<=n;i++){
  dp[i]=(pre[i]-pre[max(0,i-k)]+MOD)%MOD;
  pre[i+1]=(pre[i]+dp[i])%MOD;
}
// dp[i] phu thuoc doan [i-k..i-1] -> prefix sum; neu la min/max -> deque (muc duoi)
```

20.3 Knapsack: các biến thể

```
// 0/1: moi do vat 1 lan -> duyet w NGUOC
for(auto [w,v]:items) for(int j=W;j>=w;j--) dp[j]=max(dp[j],dp[j-w]+v);
// Vo han: duyet w XUOI (dung lai do vat nhieu lan)
for(auto [w,v]:items) for(int j=w;j<=W;j++) dp[j]=max(dp[j],dp[j-w]+v);
// Bounded (moi do vat co c ban): chia nhi phan 1,2,4,... thanh do vat 0/1
int c=count[i];
for(int x=1;c>0;x*=2){ int t=min(x,c); items.pb({t*w,t*v}); c-=t; }
// W qua lon nhung tong gia tri V nho: doi chieu dp[j]=trong luong min dat gia tri j
// Dem cach (Coin Combinations): do vat vong ngoai -> khong trung, khong phan biet thu tu
```

20.4 LIS $O(n \log n)$ và biến thể

```
// tang chat: lower_bound; KHONG giam: doi thanh upper_bound
vector<int> tails;
for(int x:a){
  auto it=lower_bound(tails.begin(),tails.end(),x);
  if(it==tails.end()) tails.pb(x); else *it=x;
}
// do dai = tails.size() (tails khong phai day con thuc)
// Truy vet / dem / cap nhat diem 1 vi tri: BIT max (muc "BIT theo gia tri" o tren)
```

20.5 DP chuỗi: wildcard ký tự * ?

```
// dp[i][j] = a[0..i) khop b[0..j) hay khong
dp[0][0]=1;
for(int j=1;j<=m;j++) if(b[j-1]=='*') dp[0][j]=dp[0][j-1];
for(int i=1;i<=n;i++) for(int j=1;j<=m;j++){
  if(b[j-1]=='*') dp[i][j]=dp[i-1][j]||dp[i][j-1]; // '*' an 0 hoac nhieu ky tu
  else dp[i][j]=dp[i-1][j-1]&&(a[i-1]==b[j-1]||b[j-1]=='?');
}
// dap an: dp[n][m] (a = xau nguon, b = mau)
```

20.6 Bitmask bổ sung

Duyệt mọi tập con của `mask` (tổng $O(3^n)$ cho toàn bộ mask):

```
for(int mask=1;mask<(1<<n);mask++)
  for(int sub=mask;sub;sub=(sub-1)&mask)
    // sub chay qua cac tap con khac rong cua mask
    // vi du: dp[mask] = min(dp[mask^sub] + cost(sub))
```

Elevator Rides (1653): $n \le 20$, thang máy tải trọng X. Trạng thái là cặp (số chuyến, tải chuyến cuối) — ưu tiên ít chuyến, rồi tải nhỏ:

```
pair<int,ll> dp[1<<20]; // {so chuyen, tai chuyen cuoi}
dp[0]={1,0};
for(int mask=1;mask<(1<<n);mask++){
  dp[mask]={INT_MAX,0};
  for(int i=0;i<n;i++) if(mask>>i&1){
    auto pr=dp[mask^(1<<i)]; pair<int,ll> c;
    if(pr.second+w[i]<=X) c={pr.first,pr.second+w[i]};
    else c={pr.first+1,w[i]};
    if(c.first<dp[mask].first||(c.first==dp[mask].first&&c.second<dp[mask].second))
      dp[mask]=c;
  }
}
// dap an: dp[(1<<n)-1].first
// Dem duong di Hamilton (1690): dp[mask][last] dem so cach, chuyen y het TSP
```

20.7 DP cửa sổ trượt bằng deque

Khi $dp[i]$ cần min/max của $dp[j]$ với j trong cửa sổ trượt, dùng deque giữ chỉ số theo thứ tự giá trị. Ví dụ Maximum Subarray Sum II (1644): tổng đoạn độ dài trong $[a, b]$ lớn nhất.

```
// ans = max(pre[i] - min(pre[j])), j trong [i-b, i-a]
deque<int> dq; ll ans=-INF64;
for(int i=a;i<=n;i++){
  while(!dq.empty()&&dq.front()<i-b) dq.pop_front();
  int j=i-a; // ung vien moi
  while(!dq.empty()&&pre[dq.back()]>=pre[j]) dq.pop_back();
  dq.pb(j);
  ans=max(ans,pre[i]-pre[dq.front()]);
}
```

20.8 Max subarray 2D — Kadane nâng cấp $O(n^3)$

```
// Tong hinh chu nhat con lon nhat trong luoi n x m
ll ans=-INF64;
for(int top=0;top<n;top++){
  vector<ll> col(m,0);
  for(int bot=top;bot<n;bot++){
    for(int j=0;j<m;j++) col[j]+=a[bot][j];
    ll cur=-INF64,best=-INF64; // Kadane tren col
    for(int j=0;j<m;j++){
      cur=max(col[j],cur+col[j]); best=max(best,cur);
    }
    ans=max(ans,best);
  }
}
```

20.9 Tree DP nhỏ: đường kính qua mỗi đỉnh

```
// h[u] = do sau lon nhat trong cay con; duong kinh = max(d1+d2) qua moi dinh
int h[N]; ll best=0;
void dfs(int u,int p){
  h[u]=0; int d1=0,d2=0;
  for(int v:g[u]) if(v!=p){
    dfs(v,u);
    if(h[v]+1>d1){ d2=d1; d1=h[v]+1; }
    else if(h[v]+1>d2) d2=h[v]+1;
  }
  h[u]=d1; best=max(best,(ll)d1+d2);
}
```

20.10 Đếm đường đi tránh vật cản (tổ hợp + đệ quy)

Đi từ $(0, 0)$ đến (N, M) chỉ sang/phải, tránh K ô cấm. Sort ô cấm, coi đích là ô cấm thứ K; $dp[i]$ = số đường tới ô i mà không dính ô cấm nào trước đó.

```
sort(obs.begin(),obs.end()); obs.pb({N,M});
vector<ll> dp(K+1);
for(int i=0;i<=K;i++){
  auto [x,y]=obs[i];
  dp[i]=Cnk(x+y,x);
  for(int j=0;j<i;j++){
    auto [a,b]=obs[j];
    if(a<=x&&b<=y) dp[i]=(dp[i]-dp[j]*Cnk(x-a+y-b,x-a))%MOD;
  }
  dp[i]=(dp[i]+MOD)%MOD;
}
// dap an: dp[K] (can nCr toi N+M, xem phu luc so hoc)
```

20.11 Broken profile DP: lấp domino 1×2 (Counting Tilings 2181)

Quét từng ô theo hàng; `mask` = m ô phía trước đã bị chiếm chỗ hay chưa. Chọn m là chiều nhỏ. $O(nm2^m)$.

```
// Lap kin luoi n x m bang domino 1x2 (n*m chan)
vector<ll> dp(1<<m,0),ndp(1<<m);
dp[0]=1;
for(int pos=0;pos<n*m;pos++){
  fill(ndp.begin(),ndp.end(),0);
  for(int mask=0;mask<(1<<m);mask++){
    ll v=dp[mask]; if(!v) continue;
    if(mask&1) ndp[mask>>1]=(ndp[mask>>1]+v)%MOD; // o da duoc lap
    else{
      // dat doc: chiem o duoi (bit m-1 cua mask moi)
      ndp[(mask>>1)|(1<<(m-1))]=(ndp[(mask>>1)|(1<<(m-1))]+v)%MOD;
      // dat ngang: chiem o ben phai (bit 0 cua mask moi)
      if(pos%m!=m-1&&!(mask&2))
        ndp[(mask>>1)|1]=(ndp[(mask>>1)|1]+v)%MOD;
    }
  }
  swap(dp,ndp);
}
// dap an: dp[0]
```